Compilers

Guillermo A. Pérez

Lecture 3: DFS Galore - Symbol table, CFG, and synthesis

TL;DR: This lecture in short

How do we make the parser generate code/an AST?

We know ANTLR will give us a derivation tree. We do not know how to get from there to the symbol table or to the output code.

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Formal language theory (languages and machines, machines and computability); Graph and tree algorithms (data structures and graph algorithms)

What skills will we obtain?

- theory: tree and graph algorithms
- practice: how to implement/construct stuff from the derivation tree

How will these skills be useful?

- It will allow you to use your parser to get a semantic representation of the input program.

Top-down syntax-guided translation

Recursive descent

Recall the Python recursive-descent parser below corresponds to a grammar with rules $S \to Ab$ and $S \to Bc \ldots$

```
1  def S():
2      n = get_next_character()
3      # If n is in First(A b Follow(S))
4      if n == 'a1' or ... or n == 'an' :
5          r = A()
6          if (!r): return False
7          n = read_next_character() # Consume
8          if (n == 'b'):
9              # compute semantic values
10             return True
11         else: return False
```

Abstract syntax tree

A definition of ASTs

It is a tree representation of the abstract syntactic structure of an input program.

- Each node of the tree denotes a construct occurring in the program.
- Why abstract? It does not represent every detail in the real syntax (grammar), only structural semantically-important details.

Examples of abstraction (w.r.t. the derivation tree)

- Constant expressions may be evaluated into a single constant instead of keeping their derivation tree.
- If-condition-then-else constructs become a single node with 3 children: the condition, the then block, the else block.
- Parentheses may be dropped if the tree preserves operation priority.

Example: binary palindromes

(1)	S	$\rightarrow$	$0S0$
(2)		$\rightarrow$	$1S1$
(3)		$\rightarrow$	$\#$

Exercise

- What is the derivation tree for 0110101#1010110?
- Give a reasonable AST for it of height 1.
- Give a grammar for real numbers, the derivation tree for 2345.222 and a reasonable AST for it.

Example: concrete vs abstract syntax tree

```c
#include <stdio.h>
int i = 5;
int f(int j) {
  for(int i = j; i < 10; i++);
  return i + 1;
}
int main() {
  return f(i + 1);
}
```

Where are we with our compiler?

Ideal compiler-development process

1. Regular expressions for the scanner are prepared
2. A parser-friendly grammar for the input language is devised
3. An AST for the input language is designed
4. YOU can do a manual traversal of the explicit derivation tree (given by ANTLR) to build the AST
5. Several AST-pass phases are introduced to, a.o., create the symbol table

An efficient data structure for ASTs

It is a tree with ordered children

But keep in mind the following:

- The tree will be built bottom-up following the derivation tree. This means nodes will have to adopt already created sub-trees.
- Recursive rules create "lists" of siblings. (See binary palindrome example.)
  - But then parent/node replacement will have to be implemented
- Some nodes have a fixed number of children (unary, binary operators in a language). Others feel more like the biblical character Abraham: unbounded number of children.

More on the AST data structure

Useful functions

- Make a node.
- A function to make nodes x and y siblings (possibly returning a dummy-rooted tree)
- A function to make node p adopt children $x_1, \ldots, x_k$

Some special node types

Variable values vs. variable locations

When a variable is used in a program it may, depending on the context, refer to

- an R-value: the value of the variable (i.e. it can appear on the RHS of an assignment)
- an L-value: the location of its contents (i.e. it can appear on the LHS of an assignment).

Dealing with locations

- Have an AST-node-type dealing with de-referencing and
- treat variable names as location names (i.e. L-values)

L-value, R-value example

```
1   x = y;    // all of
2   *x = &y; // this is
3   *x = y;   // C code
```

1. We want the location with name x to store the R-value of y (obtained by de-referencing the location with name y)
2. We want the location, whose address is the R-value of x, to store the location address with name y
3. We want the location, whose address is the R-value of x, to store the R-value of y

Tip

We can modify the grammar to recognize left and right expressions and apply the correct de-referencing, based on the above ideas, using semantic actions in the assignment production rules.

Symbol table creation

We now have an AST

We will now describe a first pass on the AST which we will use to create the symbol table.

Why symbol table(s)?

- Record symbol declarations (their type, name, scope, accessibility)
- To allow for later type-checking of the code

Goals of our symbol-table creating pass

- Create the symbol table
- Connect every symbol reference to its entry in the table

A simple data structure

Scoping

Remember: To deal with variable scoping, we need

- Either a tree of symbol tables, one per block, linked bottom-up to enable symbol searches
- or a single table with scope attributes

Every individual scope will reference their table and every table will reference tables from enclosing scopes!

Useful functions

- Open a new scope.
- Close a scope.
- Enter a new symbol: remember overloading may be desirable!
- Retrieve symbol: this may require traversing tables of enclosing scopes

A data structure for the traversal

While creating the tables

Top-down traversal of the AST to create the symbol tables makes a single pass sufficient.

- Going from top to bottom, scopes are traversed in the natural order
- Note that scopes are opened and closed in a LIFO fashion
- Hence: a stack of tables is an ideal data structure

An implicit stack

If your tree-traversal is recursively implemented:

- The call stack is implicitly simulating the table stack
- Hence: a reference to the table of the enclosing scope and a reference to the current scope's table suffice

Type descriptors

Types

One of the most important pieces of information regarding a symbol, is its type. A type can be built-in or user-created (or even objects or classes).

Built-in types

These may come from a pre-determined list of strings or be indices from a table.

User-created

These may get as intricate and complicated as the language allows. In most modern programming languages, new types inherit from other types. That is, they also have a type!

- The type of a symbol, may be itself a reference to a symbol table describing all its attributes

Search-efficient tables

Hash tables

You may want to have each symbol table be a hash table. Every declaration, usage, of a symbol will require that you look up the symbol.

- Depending on the language, different hash functions may be better

Break

- If no errors were found in the input, we now have the derivation tree induced by the parser (and the grammar)
- A first pass on the tree has allowed us to construct a symbol table with information about the types of all declared symbols
- It is useful to make AST nodes corresponding to symbol usage reference their entries in the corresponding symbol table
- Note that we can already make sure every used symbol has been declared and that no invalid re-declarations occur in the input program

For more ANTLR-specific things:
`https://github.com/antlr/antlr4/blob/master/doc/faq/parse-trees.md`

Depth-first search on the derivation tree

```
1  def DFS(G, v):
2    S = Stack()
3    S.push(v)
4    while len(S) > 0:
5      v = S.pop()
6      if v not in visited:
7        visited.append(v)
8        S.push(v)
9        for (x, y) in G.edges(v):
10         S.push(y)
11     else:
12       # process v in post-order
```

DFS-based table creation

Table creation

- Initial table (and pass current one to DFS)
- Is node v a function declaration?
- Is node v the start of a compound statement?
- Is node v a for loop?
- ...

Symbol addition

- Function headers/declarations
- Declaration statements
- For loops

Otherwise, still useful to add, per instruction, a reference to its most local symbol table.

Block-tree abstraction

Guiding principle

Nodes of the tree that correspond to statement lists get merged with children into a basic block if they are all instructions that execute in order.

DFS returns subtrees

In this case the DFS generates (in post-order) sub-trees with blocks.

Further passes

Most nodes now correspond to basic blocks, and control-flow constructs. A further pass allows to abstract the control-flow conditionals, loops, etc. into jumps and conditional jumps only! (Use block names or IDs as targets)

- For loops are best translated into while loops in an earlier pass to make the declarations and updates explicit
- Conditions are also best made explicit: factor condition out in an earlier pass
- Break, return, continue . . . require you keep a stack of contexts

Optional: control-flow graph

CFG

1. Our AST now groups instructions/statements into basic blocks
2. Jumps and conditional jumps target blocks
3. We can now make a control-flow graph in a single pass!

Some optimizations are CFG-dependant

And the use of the Phi function in a smart way requires you compute it.

Synthesis: a final DFS

Jumps and blocks

Good news: even without the CFG you are ready to generate LLVM blocks and jumps now that the tree nodes are blocks and we have simplified control-flow constructs. Make sure all your blocks end with a terminator!

Synthesis: a final DFS

Jumps and blocks

Good news: even without the CFG you are ready to generate LLVM blocks and jumps now that the tree nodes are blocks and we have simplified control-flow constructs. Make sure all your blocks end with a terminator!

Expressions and complex statements

LLVM IR (and other low-level representations) are 3-address code. So all expressions have to be simplified doing a further DFS that generates intermediate subexpression names. E.g. $x = a * b + 3$.

Synthesis

The current AST should be close enough to LLVM that every statement in every block is easy to translate to LLVM during a DFS!